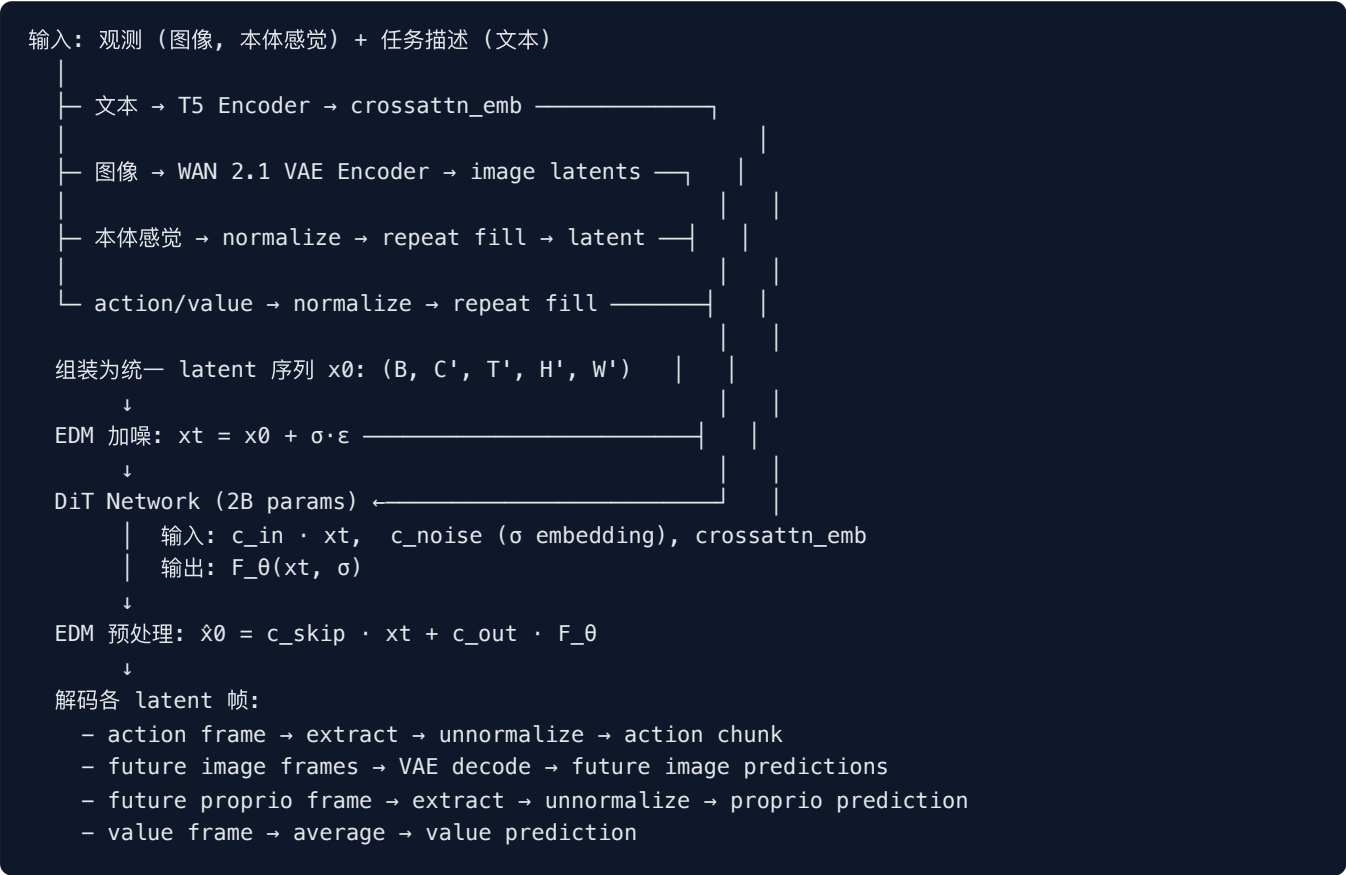


Cosmos Policy 模型设计详解

基于 paper "*Cosmos Policy: Fine-Tuning Video Models for Visuomotor Control and Planning*" (arXiv 2601.16163) 和 NVIDIA Cosmos Cookbook 的方法描述，结合代码逐模块拆解维度流转。

1. 顶层架构总览



2. WAN 2.1 VAE Tokenizer

代码: cosmos_policy/tokenizers/wan2pt1.py

属性	值	来源
latent channels (<code>z_dim</code>)	16	<code>wan2pt1.py:102</code> <code>dim=96, z_dim=16</code>
spatial compression	8×	<code>spatial_compression_factor = 8</code>
temporal compression	4×	<code>temporal_compression_factor = 4</code>
input resolution	224×224 pixel	<code>config resolution="224"</code>
latent spatial size	28×28	$224 / 8 = 28$
latent temporal formula	<code>1 + (T_pixel - 1) // 4</code>	<code>get_latent_num_frames()</code>
pixel temporal formula	<code>(T_latent - 1) * 4 + 1</code>	<code>get_pixel_num_frames()</code>

编码/解码的维度流转

编码:

```
输入视频: (B, C=3, T_pixel, H=224, W=224)
  ↓ WAN 2.1 Encoder (3D Conv + Temporal Downsample ×2)
  ↓ spatial: 224→112→56→28 (3 stages, factor 8)
  ↓ temporal: T_pixel → 1 + (T_pixel-1)//4
Latent: (B, C'=16, T_latent, H'=28, W'=28)
  ↓ 归一化: (latent - mean) / std
输出: 归一化 latent
```

解码 (推理时解码 future image predictions) :

```
Latent: (B, C'=16, T_latent, H'=28, W'=28)
  ↓ 反归一化: latent * std + mean
  ↓ WAN 2.1 Decoder
输出视频: (B, C=3, T_pixel, H=224, W=224)
```

Mean/Std: 每个通道独立, 16 维向量 (`wan2pt1.py:229-264`)。

3. Latent Frame Injection (核心创新)

3.1 设计哲学

将低维非图像数据 (action, proprio, value) 编码为与图像 latent 相同形状的帧, 直接嵌入视频 latent 序列中。

3.2 注入方法

代码: `policy_text2world_model.py:45-173` 的 `replace_latent_with_action_chunk()` 和 `replace_latent_with_proprio()`

Action 注入 (以 LIBERO 为例)

输入:

`x0: (B, C'=16, T', H'=28, W'=28)` – VAE 编码的 latent 序列
`action_chunk: (B, chunk_size=16, action_dim=7)` – GT 动作序列
`action_indices: (B,)` – action 帧在 `T'` 维度的索引

计算:

```
flat_action = action_chunk.reshape(B, 16*7) = (B, 112)
latent_elements = 16 × 28 × 28 = 12,544
num_repeats = ceil(12,544 / 112) = 113
repeated_action = flat_action.repeat(1, 113)[: , :12,544]
→ reshape to (B, 16, 28, 28)
```

输出:

`new_x0: (B, C'=16, T', H'=28, W'=28)` – action 帧被覆盖为重复的 action 值

关键: `112 << 12,544`, action 只占 latent 帧的 ~0.9%, 其余被重复填充。

Proprio 注入

输入:

`proprio: (B, proprio_dim=9)`
latent 帧大小: $16 \times 28 \times 28 = 12,544$

计算:

```
num_repeats = ceil(12,544 / 9) = 1,395
→ repeat proprio 1,395 次, 取前 12,544 个元素
→ reshape to (B, 16, 28, 28)
```

Value 注入

```
# 最简单: 直接 broadcast 填满整个 latent 帧
value: (B,) → reshape (B,1,1,1) → expand (B, C'=16, H'=28, W'=28)
```

3.3 推理时的解码

推理时从去噪后的 latent 帧中提取预测值:

- **Action:** 从 action latent 帧提取前 `chunk_size × action_dim` 个元素, reshape, 取所有重复值的平均, unnormalize
- **Future image:** 直接 VAE decode 对应帧
- **Value:** 对整个 value latent 帧取平均得到标量, unnormalize

4. Latent 序列布局

4.1 LIBERO (state_t=9)

```

帧索引:  [0]      [1]      [2]      [3]      [4]      [5]      [6]
[7]      [8]
内容:    blank  curr_proprio curr_wrist_img curr_primary_img action  future_proprio
future_wrist_img future_primary_img value
类型:    cond   cond       cond       cond       denoise  denoise      denoise
denoise  denoise
像素帧:  4张    4张       4张       4张       4张    4张      4张
4张      4张
chunk_duration = 1 blank + 8 concepts × 4 duplicates = 33 pixel frames
T_latent = 1 + (33-1)//4 = 9 latent frames

```

每个概念用 4 张相同像素帧 → VAE 编码为 1 个 latent 帧（因为 temporal_compression=4）。

4.2 RoboCasa / ALOHA (state_t=11)

```

帧索引:  [0]      [1]      [2]      [3]      [4]      [5]      [6]
[7]      [8]      [9]      [10]
内容:    blank  curr_proprio curr_wrist_img curr_primary_img curr_secondary  action
future_proprio future_wrist_img future_primary_img future_secondary  value

(RoboCasa: img2)                (RoboCasa: img2)

(ALOHA: wrist2)                  (ALOHA: wrist2)
类型:    cond   cond       cond       cond       cond       denoise  denoise
denoise  denoise  denoise  denoise  denoise
chunk_duration = 1 blank + 10 concepts × 4 duplicates = 41 pixel frames
T_latent = 1 + (41-1)//4 = 11 latent frames

```

4.3 条件帧 vs 去噪帧

```

# 配置
min_num_conditional_frames = 4 # LIBERO; 5 for RoboCasa/ALOHA
# 含义: blank + (min_num_conditional_frames - 1) 个观测帧

# condition_video_input_mask: (B, 1, T', H', W')
# = 1 表示条件帧 (无噪声, 直接替换为 GT)
# = 0 表示去噪帧 (从噪声恢复)

# sigma_conditional = 0.0 → 条件帧视为干净信号

```

在 `denoise()` 中 (`policy_video2world_model.py:394-567`):

```

# 条件帧替换
net_state_in = gt_frames/sigma_data * mask + net_state_in * (1 - mask)
# c_noise 也对条件帧做调整
c_noise = c_noise_cond * mask + c_noise * (1 - mask)

```

5. Conditioner（条件编码器）

代码: `cosmos_policy/conditioner.py`, `_src/predict2/conditioner.py`

GeneralConditioner

处理文本条件，包含 T5 text encoder。

```
输入: data_batch["caption"] = "put the soup in the basket"
    ↓ T5 Encoder
crossattn_emb: (B, seq_len=256, dim=1024) - T5-XXL hidden states
crossattn_mask: (B, seq_len=256)
```

Text2WorldCondition

```
@dataclass
class Text2WorldCondition:
    crossattn_emb: (B, seq_len, dim) # T5 文本嵌入
    data_type: DataType # VIDEO or IMAGE
    padding_mask: (B, seq_len) # 文本 padding mask
    fps: (B,) # 帧率 (可选)
```

Video2WorldCondition（扩展）

在 `Text2WorldCondition` 基础上添加：

```
gt_frames: (B, C'=16, T', H'=28, W'=28) # 条件帧的 GT latent
condition_video_input_mask: (B, 1, T', H'=28, W'=28) # 哪些帧是条件帧
```

重要改动：Cosmos Policy 的 `dropout_rate=0.0`，不做 CFG（Classifier-Free Guidance），因为文本条件不能 dropout（否则模型不遵循语言指令）。当所有 `dropout=0` 时，`uncondition=None`。

6. DiT Network（2B 参数）

代码: `_src/predict2/networks/minimal_v4_dit.py`

网络输入

```
self.net(
    x_B_C_T_H_W=net_state_in, # (B, C'=16, T', H'=28, W'=28) - c_in · xt
    timesteps_B_T=c_noise, # (B, T') - σ embedding
    crossattn_emb=..., # (B, seq_len, dim=1024) - T5 嵌入
    crossattn_emb_mask=..., # (B, seq_len) - 文本 mask
)
```

网络结构

基于 Cosmos Predict2 的 DiT (Diffusion Transformer) :

1. **Patch embedding**: 将 latent 切成 patches, 映射到 hidden dim
2. **Positional encoding**: 3D 时空位置编码
3. **Transformer blocks**: 交替的 self-attention + cross-attention 层
 - Self-attention: latent patches 之间
 - Cross-attention: latent patches attend to T5 embeddings
4. **AdaLN (Adaptive Layer Norm)**: 用 σ embedding 调制 hidden states
5. **Final linear**: 输出同形状的残差

维度变化 (以 LIBERO 为例)

```

输入: (B, 16, 9, 28, 28)
  ↓ patch embedding (2x2 spatial patches)
Flatten: (B, 9x14x14=1764, hidden_dim)
  ↓ xN Transformer blocks (self-attn + cross-attn + FFN)
Hidden: (B, 1764, hidden_dim)
  ↓ unpatch + output linear
输出: (B, 16, 9, 28, 28)

```

7. EDM 扩散框架

7.1 EDM 预处理 (Preconditioning)

代码: `policy_video2world_model.py:394-467`, 基于 EDM paper (arXiv 2206.00364)

```

# Scaling functions (EDM Eq.7)
c_skip( $\sigma$ ) =  $\sigma_{\text{data}}^2 / (\sigma^2 + \sigma_{\text{data}}^2)$ 
c_out( $\sigma$ ) =  $\sigma_{\text{data}} \cdot \sigma / \sqrt{(\sigma^2 + \sigma_{\text{data}}^2)}$ 
c_in( $\sigma$ ) =  $1 / \sqrt{(\sigma^2 + \sigma_{\text{data}}^2)}$ 
c_noise( $\sigma$ ) =  $1/4 \cdot \ln(\sigma)$ 

# Forward pass
F_0 = net(c_in · xt, c_noise)           # 网络预测
x0 = c_skip · xt + c_out · F_0          # x0 预测
ε = (xt - x0) / σ                       # noise 预测

```

7.2 加噪过程

```

# training_step:
sigma_B_T = draw_training_sigma()        # HybridEDMSDE 采样
epsilon_B_C_T_H_W = N(0, 1)              # 标准高斯噪声
mean_B_C_T_H_W, std_B_T = sde.marginal_prob(x0, sigma)
xt = mean + epsilon * std                 # 加噪

```

7.3 HybridEDMSDE（混合 Sigma 分布）

代码: `modules/hybrid_edm_sde.py`

```
# 训练时 sigma 采样 (每个 batch)
if hybrid_sigma_distribution:
    # 70% 从 log-normal 采样 (标准 EDM)
    # 30% 从 Uniform(1.0, 85.0) 采样
    distribution_choice = rand(B) < 0.7
    samples[choice] = exp(log_normal_icdf(rand())) # 70%
    samples[~choice] = uniform(1.0, 85.0)         # 30%
```

训练 vs 推理 sigma 范围对比:

	sigma_max	sigma_min
训练	200	0.01
推理	80	4

推理时使用更窄的 sigma 范围 → 更准确的预测。

7.4 采样器

代码: `modules/cosmos_sampler.py`

```
输入: x0_fn (denoiser), x_sigma_max (纯噪声)
↓ 生成时间戳序列 (rho=7 power scheduling)
↓ differential_equation_solver (2ab multistep)
↓ 迭代去噪
↓ 额外 sample_clean 步: 再过一次 denoiser 得到完全去噪结果
输出: 去噪后的 latent
```

推理时默认 `num_steps=5` (action), `num_steps=1` (future state + value)。

8. 训练目标的条件方案

8.1 三种训练模式

论文中 Balanced Batch Splitting: 每个 batch 按比例混合三种样本。

模式 1: Policy Prediction (~50% batch)

```
条件: [s] → 当前状态 (proprio + images)
目标: [a, s', V(s')]
Loss: 只在 action + future state + value 帧计算
```

模式 2: World Model Prediction (~25% batch)

条件: $[s, a] \rightarrow$ 当前状态 + action
 目标: $[s', V(s')]$
 Loss: 只在 future state + value 帧计算
 实现: 将 action 帧的 condition mask 设为 1 (干净条件)

模式 3: Value Function Prediction (~25% batch)

条件: $[s, a, s'] \rightarrow$ 当前状态 + action + 未来状态
 目标: $[V(s')]$
 Loss: 只在 value 帧计算
 实现: 将所有帧的 condition mask 设为 1, 只有 value 帧为 0

8.2 Loss Mask 的实现

```
# compute_loss_with_epsilon_and_sigma():
final_mask = ones(B, T') # 初始全 1

# 1) Value prediction: 只保留 value 帧
if mask_current_state_action_for_value_prediction:
    mask = zeros(B, T')
    mask[batch, value_idx] = 1 # 只有 value 帧有 loss

# 2) Policy + World Model: 按样本类型选帧
if mask_loss_for_action_future_state_prediction:
    mask = zeros(B, T')
    # Demo 样本: action 帧 = 1
    mask[demo_batch, action_idx] = 1
    # World model 样本: future state 帧 = 1
    mask[world_batch, future*_idx] = 1
    # Value 样本: value 帧 = 1
    mask[value_batch, value_idx] = 1

# Action loss 加权
if action_loss_multiplier > 1:
    final_mask[batch, action_idx] *= multiplier

# 最终 loss
edm_loss = MSE(x0, pred.x0) * weight_per_sigma * final_mask
```

8.3 Loss 维度

pred_mse: (B, C'=16, T', H'=28, W'=28) - 逐元素 MSE
 edm_loss: (B, C'=16, T', H'=28, W'=28) - 加权 MSE
 kendall_loss: scalar - 最终 loss (mean 或 sum)

额外记录的 per-component losses (MSE + L1):

- demo_sample_action_mse/l1_loss
- demo_sample_future_proprio_mse/l1_loss
- demo_sample_future_image_mse/l1_loss
- demo_sample_value_mse/l1_loss
- world_model_sample_future*_mse/l1_loss
- value_function_sample_value_mse/l1_loss

8.4 收敛时的参考 Loss 值

Loss	典型范围
Action L1	~0.010–0.015
Future proprio L1	~0.007
Future image latent L1	~0.05–0.09
Value L1	~0.007

9. 推理管线维度追踪（LIBERO 示例）

Step 1: 构建观测 data_batch

```
observation = {
    "primary_image": (H=224, W=224, C=3),      # 第三人称图像
    "wrist_image": (H=224, W=224, C=3),        # 腕部图像
    "proprio": (proprio_dim=9,),                # 本体感觉
}
task_description = "put both the alphabet soup and the tomato sauce in the basket"
```

Step 2: VAE 编码

```
# 4 张相同图像 → 1 个 latent 帧 (temporal_compression=4)
primary_images = repeat(primary_image, 4 times) # (1, 3, 4, 224, 224)
primary_latent = vae.encode(primary_images)      # (1, 16, 1, 28, 28)

wrist_images = repeat(wrist_image, 4 times)     # (1, 3, 4, 224, 224)
wrist_latent = vae.encode(wrist_images)          # (1, 16, 1, 28, 28)

proprio = normalize(p proprio)                  # (1, 9)
proprio_latent = repeat_to_fill(proprio)         # → (1, 16, 1, 28, 28) via injection
```

Step 3: 组装 latent 序列

```
# LIBERO: 9 个 latent 帧
x0 = zeros(1, 16, 9, 28, 28)
x0[:, :, 0, :, :] = blank_frame
x0[:, :, 1, :, :] = proprio_latent      # 注入
x0[:, :, 2, :, :] = wrist_latent       # VAE 编码
x0[:, :, 3, :, :] = primary_latent     # VAE 编码
x0[:, :, 4, :, :] = blank (action, 待去噪)
x0[:, :, 5, :, :] = blank (future proprio, 待去噪)
x0[:, :, 6, :, :] = blank (future wrist, 待去噪)
x0[:, :, 7, :, :] = blank (future primary, 待去噪)
x0[:, :, 8, :, :] = blank (value, 待去噪)

# condition_video_input_mask
mask = zeros(1, 1, 9, 28, 28)
mask[:, :, 0:4, :, :] = 1 # blank + proprio + wrist + primary = 条件帧
```

Step 4: 加噪

```
sigma_max = 80
x_sigma_max = randn(1, 16, 9, 28, 28) × 80 # 纯噪声
```

Step 5: 去噪 (5 步 action)

```
# 每步:
x0 = denoise(xt, sigma, condition)
→ DiT network forward
→ 条件帧替换为 GT (mask 为 1 的帧)
→ 去噪帧被预测

# 5 步后 + 1 步 sample_clean
samples: (1, 16, 9, 28, 28)
```

Step 6: 解码预测

```
# Action 解码
action_latent = samples[:, :, 4, :, :]      # (1, 16, 28, 28)
flat = action_latent.flatten()              # (12,544,)
action_values = flat[:chunk_size×action_dim] # 前 112 个值
action_values = average_over_duplicates()    # 取重复值的平均
actions = unnormalize(action_values)         # (16, 7)

# Future image 解码
future_primary_latent = samples[:, :, 7:8, :, :] # (1, 16, 1, 28, 28)
future_primary = vae.decode(future_primary_latent) # (1, 3, 1, 224, 224)

# Value 解码
value_latent = samples[:, :, 8, :, :]       # (1, 16, 28, 28)
value = value_latent.mean()                 # 标量
value = unnormalize(value)
```

10. Best-of-N Planning

流程

1. 生成 N 个 action 候选 (不同随机种子)

2. 对每个候选:

a. `get_action_prediction()` → action chunk + future state + value

3. 选 value 最高的 action chunk

4. 执行 action chunk 的前 `num_open_loop_steps` 个动作

5. 滑窗推进到下一时刻

维度

```
# N 个并行候选
for seed in [seed, seed+1, ..., seed+N-1]:
    action_return_dict = get_action_prediction(seed=seed)

# actions: (chunk_size, action_dim) e.g., (16, 7) for LIBERO
# value: scalar
# future_images: dict of (H, W, C) arrays

# 选择
best = max(candidates, key=lambda x: x.value)
action_queue.extend(best.actions[:num_open_loop_steps])
```

11. 平台维度汇总

维度	LIBERO	RoboCasa	ALOHA
state_t (latent frames)	9	11	11
conditional_frames	4	5	5
chunk_duration (pixel)	33	41	41
action_dim	7	7	14
proprio_dim	9	9	14
chunk_size	16	32	50
action_flat_dim	112	224	700
latent_frame_elements	12,544	12,544	12,544
action 占 latent 比	0.89%	1.79%	5.58%
camera views	1 wrist + 1 primary	1 wrist + 2 primary	2 wrist + 1 primary
gamma (value)	0.99	0.99	0.998

12. 关键数值参数

模型规模

- 参数量: 2B (Cosmos-Predict2-2B)
- 基础模型: `nvidia/Cosmos-Predict2-2B-Video2World/model-480p-16fps.pt`
- **Fine-tune** 方式: 单阶段 post-training, 不修改架构

训练超参

参数	LIBERO	RoboCasa	ALOHA
Global batch size	1920	800	200
Gradient steps	40K	45K	50K
Learning rate	1e-4	1e-4	1e-4
LR decay step	30K	30K	20K
Optimizer	FusedAdamW	FusedAdamW	FusedAdamW
Distributed	FSDP	FSDP	FSDP
GPU 数量	64 H100	32 H100	8 H100

推理超参

参数	值
denoising steps (action)	5
denoising steps (future state)	1
denoising steps (value)	1
solver	2ab multistep
guidance	1.5 (but no CFG since dropout=0)
sigma_max (inference)	80
sigma_min (inference)	4